

Query builder

```
$builder = DB::table('users')->where('name', 'John');

$all = $builder->get($columns = ['*']);
$all = $builder->select($columns = ['*']);
$first = $builder->first($columns = ['*']);

$byId = $builder->find($id);
$value = $builder->value($column);
$pluck = $builder->pluck($column, $key = null);

$bool = $builder->exists();
$bool = $builder->doesntExist();

$select = $query->addSelect($column)->get();
$distinct = $builder->distinct()->get();

$users = $builder->count();
$price = $builder->max('price');
$price = $builder->avg('price');

// Для обновления используйте chunkById()
$builder->chunk(100, function ($users) {
    foreach ($users as $user) {/** */}
    return false; // остановить обработку дальнейших фрагментов
});

// Для обновления используйте lazyById()
$builder->lazy()->each(function ($user) {/** */});

$builder = $builder
// WHERE
->where($columns, '=', 100)
->where($columns, 100)
->where([
    [$columns, 'like', '1'],
    [$columns, '<>', '1'],
])
->where(function ($query) {
    $query->select('type')
        ->from('membership')
        ->whereColumn('membership.user_id', 'users.id')
        ->limit(1);
}, 'Pro')
->orWhere('name', 'John')
->orWhere(function($query) {
    $query->where('name', 'Abigail')
        ->where('votes', '>', 50);
})
```

```

->whereFullText('bio', 'web developer')
->whereNot('name', 'John')
->whereNotNull('updated_at')
->whereNull('last_name')
->whereIn('id', [1, 2, 3])
->whereRaw('price > IF(state = "TX", ?, 100)', [200])
->whereColumn('first_name', 'last_name'
)->whereColumn([
    ['first_name', '=', 'last_name'],
    ['updated_at', '>', 'created_at'],
])
->whereExists(function ($query) {
    $query->select(DB::raw(1))
        ->from('orders')
        ->whereColumn('orders.user_id', 'users.id');
})

```

```

->whereDate('created_at', '2016-12-31')
->whereMonth('created_at', '12')
->whereDay('created_at', '31')
->whereYear('created_at', '2016')
->whereTime('created_at', '=', '11:20:45')

```

// JSON WHERE:

```

->where('preferences->dining->meal', 'salad')
->whereJsonContains('options->languages', 'en')
->whereJsonLength('options->languages', 0)

->whereBetween('votes', [1, 100])
->whereNotBetween('votes', [1, 100])
->whereBetweenColumns('weight', ['min_weight', 'max_weight'])
->whereNotBetweenColumns('weight', ['min_weight', 'max_weight'])

```

// WHEN (not WHERE)

```

->when($role, function ($query, $role) {
    $query->where('role_id', $role);
})

->when($sortByVotes, function ($query, $sortByVotes) {
    $query->orderBy('votes');
}, function ($query) {
    $query->orderBy('name');
})

```

// Ordering

```

->orderBy('name', 'DESC')
->inRandomOrder()
->reorder()
->latest()

```

// Grouping

```

->groupBy('account_id')

->having('account_id', '>', 100)
->havingBetween('number_of_orders', [5, 15])
->havingRaw('SUM(price) > ?', [2500])

// Limit & Offset
->skip(10)->take(5)
->offset(10)->limit(5)

->join('contacts', 'users.id', '=', 'contacts.user_id')
->leftJoin('posts', 'users.id', '=', 'posts.user_id')
->rightJoin('posts', 'users.id', '=', 'posts.user_id')
->crossJoin('colors')
->join('contacts', function ($join) {
    $join->on('users.id', '=', 'contacts.user_id')
    ->orOn(/* ... */);
})

// UNION
->union($builder->whereNull('first_name'))

->get();

// Отладка
$builder->where('votes', '>', 100)->dd();
$builder->where('votes', '>', 100)->dump();

$builder->insert([
    ['email' => 'picard@example.com', 'votes' => 0],
    ['email' => 'janeway@example.com', 'votes' => 0],
]);

// INSERT
$builder->insertOrIgnore([/* */]);
$builder->insertUsing([/* */], $subQuery); // использ. подзапрос
$builder->insertGetId([/* */]); // после вставки получи ID

// UPSERT (вставит несуществующее или обновит, если уже есть)
$builder->upsert(
    [
        ['departure' => 'Oakland', 'destination' => 'San Diego', 'price' => 99],
        ['departure' => 'Chicago', 'destination' => 'New York', 'price' => 150]
    ],
    ['departure', 'destination'],
    ['price']
);

```

```
// UPDATE
$countUpdated = $builder->where('id', 1)
    ->update(['votes' => 1]);

$countUpdated = $builder->where('id', 1)
    ->updateOrCreate(['votes' => 1]);

// Updating JSON Columns
$countUpdated = $builder->where('id', 1)
    ->update(['options->enabled' => true]);

// Increment & Decrement
$builder->increment('votes');
$builder->increment('votes', 5);
// обновить дополнительно поле "name"
$builder->increment('votes', 1, ['name' => 'John']);

$builder->decrement('votes');
$builder->decrement('votes', 5);

// DELETE
$countDeleted = $builder->delete();
$builder->truncate(); // обнулить всю таблицу

// Pessimistic Locking (предотвращает изменение выбранных строк до конца операции)
$builder->sharedLock()->get();
$builder->lockForUpdate()->get();
```